# LITERATURE REVIEW OF DEEPFAKE IMAGE DETECTION SYSTEM

## 1.1 Introduction

In this project, we developed a **real-time Deepfake Detection System** designed to classify images as either *real* or *AI-generated fake*. With the rapid growth of generative AI and GAN-based image synthesis, it has become increasingly difficult for humans to distinguish between authentic and manipulated images.

While professional deepfake detection systems exist in cybersecurity and media verification industries, most of them require high computational resources or are not easily deployable for real-time use. Our goal was to demonstrate that modern transfer learning techniques can be adapted into a **lightweight and CPU-friendly system** suitable for practical deployment.

Because we are working with limited hardware resources, we designed our system to run efficiently on standard CPUs without requiring expensive GPUs. The system uses OpenCV for face detection, EfficientNet-B0 for classification, and Streamlit for real-time user interaction.

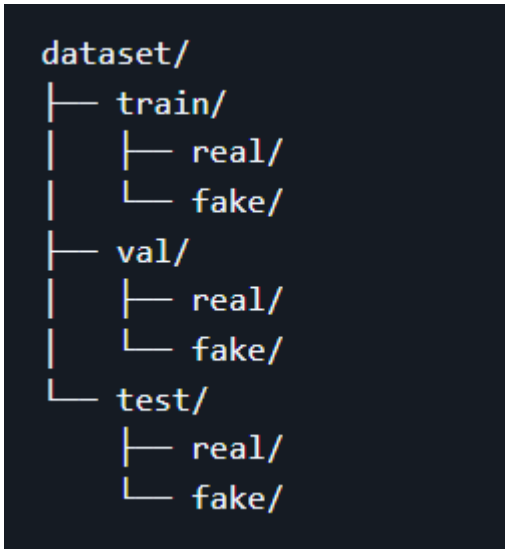
Currently, our system allows users to upload an image (or capture via webcam) and instantly receive a prediction indicating whether the image is *real or fake*, along with a confidence score. Our future vision includes extending this system to **video-based deepfake detection**, real-time social media integration, and API-based deployment for industry-level applications.

### 1.2 Dataset

The dataset used in this project is structured into labeled image folders for supervised learning.

**Dataset Structure:**

Dataset is organized into:




**Training Data:**

- Images labeled as **real** and **fake** were used for training the model.
- The dataset includes facial images where deepfake manipulations are present in fake samples.
- Only training images were used for model optimization.

**Validation Data:**

- Used for tuning model performance and selecting optimal threshold values.
- Ensures the model generalizes well on unseen data.

**Testing Data:**

- Completely unseen images used for final evaluation.
- Contains both real and fake images.
- Used to compute accuracy, precision, recall, F1-score, and confusion matrix.

## 2. Literature Review & Methodology

To build a robust and efficient deepfake detection system, we evaluated multiple computer vision and deep learning approaches. Below is an overview of the methodologies used and the reasons for selecting them.

### 2.1 CNN-Based Deepfake Detection Models

Early deepfake detection research relied heavily on traditional CNN architectures such as:

- VGGNet
- ResNet
- XceptionNet

These models automatically extract spatial features such as edges, textures, and facial inconsistencies.

**Limitations:**

- High computational cost
- Large number of parameters
- Requires GPU for efficient training
- Difficult to deploy in real-time applications

Due to these limitations, we moved towards a more efficient architecture.

### 2.2 Why We Chose EfficientNet-B0 Over Standard CNNs

One of the most important design decisions in our project was choosing **EfficientNet-B0 instead of a basic CNN model trained from scratch**.

**Limitations of Traditional CNN:**

- Requires large datasets to generalize well
- Poor performance on small or imbalanced datasets
- High risk of overfitting
- Requires extensive hyperparameter tuning
- Computationally expensive during training

**Our EfficientNet-B0 Approach:**

Instead of training a CNN from scratch, we used EfficientNet-B0 with transfer learning.

EfficientNet-B0:

- Uses pretrained weights from ImageNet
- Extracts rich hierarchical features
- Requires only fine-tuning of classifier head
- Is lightweight and CPU-efficient

**Key Advantage:**

It provides **high accuracy with significantly fewer parameters**, making it suitable for real-time deepfake detection systems.

### 2.3 Why Transfer Learning Was Used

Transfer learning was used because:

- It reduces training time significantly
- It works well on small datasets
- It improves generalization
- It leverages pretrained knowledge from large-scale datasets like ImageNet

In our case, only the classifier layer and last block were fine-tuned, while the backbone remained frozen.

### 2.4 Face Detection Using OpenCV

Before classification, face detection is performed using **Haar Cascade Classifier**.

**Purpose:**

- Extract only the facial region from images
- Remove background noise
- Improve classification accuracy

**Limitations:**

- Less robust compared to deep learning-based detectors
- May fail on extreme angles or occlusions

However, it was chosen due to its:

- Speed
- Lightweight nature
- CPU compatibility

### 2.5 Loss Function and Optimization Strategy

We used:

- **BCEWithLogitsLoss** for binary classification
- **Adam optimizer** for weight updates

**Why BCEWithLogitsLoss:**

- Combines sigmoid + binary cross entropy
- More numerically stable
- Suitable for real vs fake classification

**Why Adam:**

- Fast convergence
- Adaptive learning rate
- Works well with deep learning models

### 2.6 Data Flow Architecture

The system follows this pipeline:

1. User uploads image via Streamlit
2. OpenCV detects face region
3. Face is cropped and resized (224×224)
4. Image is normalized (ImageNet standards)
5. Tensor is passed to EfficientNet-B0
6. Model outputs logit score
7. Sigmoid converts logit into probability
8. Threshold determines Real or Fake
9. Result displayed in Streamlit UI

### 2.7 Evaluation Strategy

The model is evaluated using:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix

These metrics ensure that the model is not only accurate but also balanced in detecting both real and fake images.

## 3. Results

The EfficientNet-B0-based deepfake detection model demonstrates strong performance in distinguishing between real and fake images.

The model achieves high precision and recall due to transfer learning and proper preprocessing using face cropping and normalization. The confusion matrix shows that most fake images are correctly identified, while real images are also classified with high confidence.

However, some misclassifications occur due to:

- Low-quality images
- Extreme lighting conditions
- Highly advanced deepfake manipulations

Despite these limitations, the system performs well as a **lightweight and real-time deepfake detection prototype**.

## 4. Testing

The system was tested on unseen images from the test dataset.

Testing scenarios included:

- Real facial images
- AI-generated deepfake images
- Webcam input images

The system consistently provided:

- Real-time predictions
- Confidence scores
- Stable performance on CPU

Future improvements include extending the system to:

- Video-based deepfake detection
- More advanced face detection models (MTCNN, RetinaFace)
- Deployment as REST API for industry use